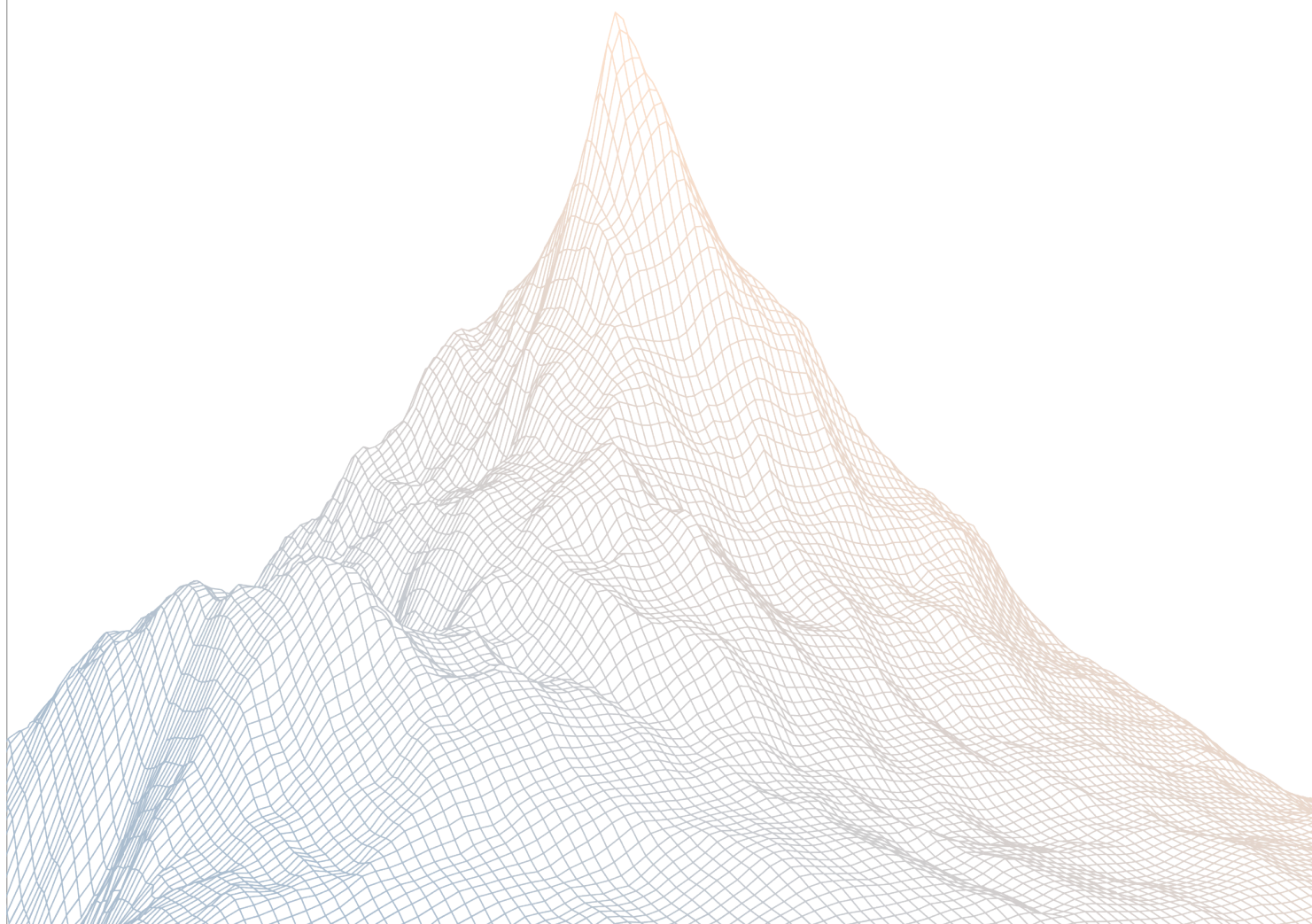


Cosmos Labs

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

May 4th to May 7th, 2026

AUDITED BY:

Bernd
Christian Vari

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Cosmos Labs	4
2.2	Scope	4
2.3	Audit Timeline	6
2.4	Issues Found	6
3	Findings Summary	6
4	Findings	7
4.1	Medium Risk	8
4.2	Low Risk	12
4.3	Informational	23

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Cosmos Labs

Cosmos powers the largest network of vertically integrated Layer 1 blockchains in the world: 200+ blockchains use the Cosmos Stack in production. Cosmos Labs is responsible for developing new software and products for the Cosmos Stack and its customers, building and maintaining customer relationships, growing the adoption of our stack, and fostering the community of independent builders and end users using our open-source products.

2.2 Scope

The engagement involved a review of the following targets:

Target	ibc-attestor
---------------	--------------

Repository	https://github.com/cosmos/ibc-attestor/
-------------------	---

Commit Hash	7819bfa88053781ce96db66a8961fbbc800eec35
--------------------	--

Files	adapter/adapter/cosmos.rs adapter/adapter/evm.rs adapter/adapter/mod.rs adapter/adapter/retry.rs adapter/adapter/solana.rs signer/signer/local.rs signer/signer/mod.rs signer/signer/remote.rs rpc/rpc/attestor.rs attestation.rs attestation_payload.rs
--------------	--

Target Mitigation Review

Repository <https://github.com/cosmos/ibc-attestor/>

Commit Hash 56752807c4e07ab30f0046d0b01348bf6569b968

Files

- adapter/adapter/cosmos.rs
- adapter/adapter/evm.rs
- adapter/adapter/mod.rs
- adapter/adapter/retry.rs
- adapter/adapter/solana.rs
- signer/signer/local.rs
- signer/signer/mod.rs
- signer/signer/remote.rs
- rpc/rpc/attestor.rs
- attestation.rs
- attestation_payload.rs

2.3 Audit Timeline

May 4, 2026	Audit start
May 7, 2026	Audit end
June 19, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	6
Informational	2
Total Issues	10

3

Findings Summary

ID	Description	Status
M-1	Remote signer opens a new TCP connection on every signing request	Resolved
M-2	Failed Cosmos receipt queries are signed as valid non-membership proofs	Resolved
L-1	Overlong Solana client IDs can panic packet attestation requests	Resolved
L-2	Remote signer signature components not individually length-validated	Resolved
L-3	Cosmos adapter returns Code::Internal for already-received packets	Resolved
L-4	Remote signer transport does not enforce TLS	Resolved
L-5	Add a non-empty password option for local keystores	Resolved
L-6	Unbounded packet batch size in packet_attestation amplifies chain RPC load	Resolved
I-1	Empty packet list bypasses commitment validation and reaches the signer	Acknowledged
I-2	Caller-controlled input errors are returned as internal service failures	Resolved

4

Findings

4.1 Medium Risk

A total of 2 medium risk findings were identified.

[M-1] Remote signer opens a new TCP connection on every signing request

SEVERITY: Medium

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [apps/ibc-attestor/src/signer/remote.rs#L50-L62](#)
- [apps/ibc-attestor/src/signer/remote.rs#L78-L106](#)

Description

`RemoteSigner::sign()` calls `create_client()` on every invocation. `create_client()` establishes a new TCP (and optionally TLS) connection to the remote signer endpoint, then issues two sequential gRPC round-trips — `GetWallet` followed by `Sign` — before tearing the channel down. No connection is cached or pooled across signing requests.

```
async fn sign(&self, message: &[u8]) → Result<Signature, SignerError> {
    let mut client = self.create_client().await?; // new TCP connect every
    time
    client.get_wallet(wallet_request).await?;      // round-trip 1 –
    unnecessary
    client.sign(request).await?;                  // round-trip 2 – actual
    signing
}
```

The `GetWallet` call is redundant: the wallet ID is static configuration, and `self.wallet_id` could be used directly in `SignRequest.wallet_id` without consulting the signer.

The combination produces three failure modes:

Signer temporarily unreachable. Every in-flight attestation request reaches the signing step and blocks inside `connect().await` for up to 30 seconds (the configured timeout). With no concurrency cap on attestation requests and no circuit-breaker, blocked tasks

accumulate. At 30 seconds per blocked task, 100 concurrent requests hold 100 tasks simultaneously, exhausting the Tokio thread pool and making the attester unresponsive for the full outage window.

File descriptor exhaustion under sustained load. Short-lived TCP connections accumulate in `TIME_WAIT` (default 60 seconds on Linux). At 50 signing requests per second, connections enter `TIME_WAIT` faster than they age out: $50 \text{ sockets/s} \times 60 \text{ s} = \sim 3,000$ open sockets at steady state, exceeding the default per-process file descriptor limit of 1,024. Once the limit is reached, `connect()` fails for all subsequent signing requests.

AWS KMS quota consumption. The redundant `GetWallet` call doubles the effective RPC rate against the KMS endpoint, halving the attestation throughput before per-account request quotas are reached.

Recommendations

We recommend establishing the gRPC channel once at construction time and reusing it for all signing requests.

Cosmos: Resolved with [@b44ee012fd ...](#)

Zenith: Verified.

[M-2] Failed Cosmos receipt queries are signed as valid non-membership proofs

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

Target

- [apps/ibc-attestor/src/adaptor/cosmos.rs#L136-L173](#)
- [apps/ibc-attestor/src/rpc/attestor.rs#L297-L345](#)

Description:

The receipt-path helper `v2_packet_receipt` decodes `abci_query` responses without inspecting the ABCI application code:

```
279: let res = self
280:     .abci_query(
281:         Some("/ibc.core.channel.v2.Query/PacketReceipt".to_string()),
282:         request.encode_to_vec(),
283:         Some(height.try_into()?),
284:         false,
285:     )
286:     .await?;
287:
288: QueryPacketReceiptResponse::decode(res.value.as_slice())
289:     .map_err(|e| anyhow::anyhow!("Failed to decode
    QueryPacketReceiptResponse: {e}"))
```

Empty value decodes to the proto default `received = false`. `get_receipt_commitment()` and `handle_receipt_commitment()` then convert that into a signed zero commitment, which the source chain accepts as a non-membership proof during timeout verification:

`get_receipt_commitment()`:

```
163: // Packet was received
164: if response.received {
165:     error!("packet was already received, cannot timeout");
166:     Err(AttestationAdapterError::CommitmentError(format!(
167:         "Packet seq={sequence} was already received, cannot timeout",
```

```

168:     )))
169:   } else {
170:     debug!("receipt commitment not found (packet not received)");
171:     Ok(None)
172:   }

```

handle_receipt_commitment():

```

320: // The expected commitment is empty commitment (for timeout proofs)
321: // so flag when commitment exists
322: commitment.map_or_else(
323:   || {
324:     debug!("receipt commitment validated (zero/non-existent as
expected)");
325:     Ok(IAttestationMsgs::PacketCompact {
326:       path: keccak256(commitment_path),
327:       commitment: [0; 32].into(),
328:     })
329:   },
330:   |commit| {
331:     error!(
332:       actual = %hex::encode(commit),
333:       "receipt commitment should be zero but found non-zero value"
334:     );
335:     Err(AttestorError::InvalidCommitment {
336:       reason: format!(
337:         "Receipt commitment should be zero for client_id={}
seq={} got 0x{}",
338:         client_id,
339:         sequence,
340:         hex::encode(commit)
341:       ),
342:     })
343:   },
344: )

```

A transient app-state unavailability can be exploited to forge a timeout proof for a packet.

Recommendations:

Consider rejecting any ABCI response with a non-zero application code.

Cosmos: Resolved with [@72748c29a1 ...](#)

Zenith: Verified.

4.2 Low Risk

A total of 6 low risk findings were identified.

[L-1] Overlong Solana client IDs can panic packet attestation requests

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [apps/ibc-attestor/src/adapters/solana.rs#L139-L148](#)
- [programs/solana/packages/solana-ibc-types/src/router.rs#L272-L309](#)
- [anza-xyz/solana-sdk/pubkey/src/lib.rs](#)

Description:

packet_attestation accepts user-provided ABI-encoded packets and decodes them without validating the semantic length of the packet client identifiers. The packet handlers then pass sourceClient to the adapter for packet commitments, and destClient for acknowledgements and receipts.

For Solana, the adapter uses that decoded client ID to derive the packet commitment, acknowledgement, or receipt PDA. The solana-ibc-types PDA helpers pass client_id.as_bytes() directly as one PDA seed and call Pubkey::find_program_address. Solana PDA seeds are limited to 32 bytes each, and this API panics when a seed exceeds that limit. Since the helpers return (Pubkey, u8) instead of a Result, the adapter cannot convert this failure into an AttestorError once an oversized client ID reaches PDA derivation.

A caller can therefore submit a Solana packet attestation request with an overlong sourceClient for Packet, or an overlong destClient for Ack and Receipt. The request panics before any Solana RPC lookup and before the service can return a controlled gRPC error.

Under the default Rust unwind behavior, this should unwind only the task handling the malformed request rather than stop the tonic listener.

Recommendations:

Consider validating Solana packet client identifiers before PDA derivation and rejecting identifiers longer than Solana's per-seed PDA limit.

Cosmos: Resolved with [@457196ca8d ...](#)

Zenith: Verified.

[L-2] Remote signer signature components not individually length-validated

SEVERITY: Low

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [apps/ibc-attestor/src/signer/remote.rs#L119-L141](https://github.com/cosmos/cosmos-sdk/blob/master/app/ibc-attestor/src/signer/remote.rs#L119-L141)

Description

The remote signer assembles the 65-byte ECDSA signature by concatenating the r, s, and v byte fields from the gRPC response, then validates only the total length:

```
let signature_bytes: Vec<_> = match signature {
    RecoverableSignature(recoverable) => {
        [recoverable.r, recoverable.s, recoverable.v]
            .into_iter()
            .flatten()
            .collect()
    }
    ...
};

if signature_bytes.len() != 65 {
    return Err(SignerError::InvalidSignature(...));
}

Signature::try_from(signature_bytes.as_slice())
```

The individual lengths of r, s, and v are never checked. A signer returning r = 33 bytes, s = 31 bytes, v = 1 byte satisfies 33 + 31 + 1 = 65 and passes the check. `Signature::try_from` then interprets the 65-byte slice as [r: bytes[0..32], s: bytes[32..64], v: bytes[64], silently misaligning the components. The attestor returns ok while the resulting signature is scrambled and will fail on-chain verification.

The error message on line 128 also contains a typo: "expected resoverable signature" should read "expected recoverable signature".

Recommendations

We recommend validating each component individually before concatenation and using a fixed-layout copy instead of a variable-length flatten.

Cosmos: Resolved with [@c424c76ade ...](#)

Zenith: Verified.

[L-3] Cosmos adapter returns Code::Internal for already-received packets

SEVERITY: Low

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [apps/ibc-attestor/src/adapter/cosmos.rs#L163-L168](#)
- [apps/ibc-attestor/src/rpc/attestor.rs#L297-L318](#)

Description

The three adapters handle the "receipt already exists" case for `CommitmentType::Receipt` inconsistently. When a receipt exists on an EVM or Solana chain, the adapters return `Ok(Some(commitment_bytes))`, which `handle_receipt_commitment` in `attestor.rs` receives and maps to `Code::InvalidArgument` — the correct code for a caller-visible protocol rejection.

The Cosmos adapter breaks this contract. When `response.received == true` it returns an `Err` directly:

```
// adapter/cosmos.rs:163-168
if response.received {
    Err(AttestationAdapterError::CommitmentError(
        format!("Packet seq={sequence} was already received, cannot
        timeout"),
    ))
}
```

This error propagates via `?` before `handle_receipt_commitment` can run, and the `CommitmentError` variant maps to `Code::Internal` — a code that signals a server-side fault.

No attestation is incorrectly produced in either path; both correctly refuse to sign a timeout for an already-received packet. The impact is limited to observability and client behaviour:

1. **False health alerts.** Monitoring that fires on `Code::Internal` spikes will alert on this as an infrastructure failure whenever a relayer requests a timeout proof for a packet already received on Cosmos.
2. **Unnecessary retries.** gRPC clients that retry transient `Code::Internal` errors will replay

the same request repeatedly, adding load to the attester and its RPC endpoint for a condition that will never resolve.

3. **Adapter contract violation.** The `get_commitment` function's contract is `Ok(None)` = not found, `Ok(Some)` = found. The Cosmos receipt adapter breaks this by returning `Err` for the "found" case, making the adapters non-interchangeable from a reasoning standpoint.

Recommendations

We recommend aligning the Cosmos receipt adapter with EVM and Solana by returning `Ok(Some(...))` when the receipt exists. If the actual receipt bytes are not available in the response, a non-zero sentinel value is sufficient to trigger the correct error path in `handle_receipt_commitment`.

Cosmos: Resolved with [@c9de3846cc ...](#) and [@56752807c4 ...](#)

Zenith: Verified.

[L-4] Remote signer transport does not enforce TLS

SEVERITY: Low

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [apps/ibc-attestor/src/signer/remote.rs#L53-L58](#)

Description

The gRPC endpoint to the remote signer is constructed without any TLS configuration or URL-scheme validation:

```
let channel = Endpoint::from_shared(self.endpoint.to_string())
    .map_err(|e| SignerError::ConnectionError(e.to_string()))?
    .timeout(Duration::from_secs(30))
    .connect()
    .await
    .map_err(|e| SignerError::ConnectionError(e.to_string()))?;
```

No call to `.tls_config(...)` is present and the code accepts any URL scheme, including `http://`. If the operator configures the endpoint without TLS — which is common on internal networks where encryption is assumed but not enforced by the client — the 33-byte signing input and the resulting ECDSA signature are transmitted in plaintext.

The remote signer is described as being based on the Cosmos Labs remote signer using AWS KMS. AWS KMS SDK connections are always TLS-encrypted, but the gRPC sidecar/proxy implementing `signerservice.proto` is what this finding concerns — that intermediate component may run on an internal network segment where plaintext is treated as acceptable.

A network-level attacker on the path between the attestor and the remote signer can:

1. **Substitute the message being signed.** The signer has no way to detect that `RecoverableMessage.message` was altered in transit. A MITM can replace the inner hash with a hash of attacker-chosen data, causing the signer to produce a valid ECDSA signature over false state. If the attacker controls paths to `m` of the `n` attestors, they reach quorum and can submit fraudulent light client proofs.
2. **Impersonate the signer endpoint.** Without certificate verification, an on-path attacker can respond to `GetWallet` and `sign` RPCs with crafted responses, making the attestor

return signatures under the belief it communicated with the legitimate KMS.

Recommendations

We recommend enforcing TLS at the endpoint level and validating the URL scheme at configuration load time

Cosmos: Resolved with [@1a5d38376c ...](#)

Zenith: Verified.

[L-5] Add a non-empty password option for local keystores

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [apps/ibc-attestor/src/signer/local.rs#L44-L76](#)
- [libs/ethereum/keys/src/signer_local.rs#L7-L24](#)

Description:

The local signer loads its private key through the shared keystore helper. That helper decrypts keystores with an empty password and also writes new keystores with an empty password.

This makes the keystore file itself the effective private key. Anyone who gains read access to the file via backups, container images, mounted volumes, CI artifacts, a host compromise, or operator workstations can decrypt the attestor key without a separate secret.

Recommendations:

Consider adding the option to use an encrypted private key with a non-empty password.

Cosmos: Resolved with [@d3c8cc35f6 ...](#)

Zenith: Verified.

[L-6] Unbounded packet batch size in `packet_attestation` amplifies chain RPC load

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [apps/ibc-attestor/src/rpc/attestor.rs#L108-L114](#)
- [apps/ibc-attestor/src/lib.rs#L48-L55](#)

Description:

`packet_attestation` accepts a `PacketAttestationRequest` with a `packets: Vec<Vec<u8>>` field of arbitrary length. There is no per-request cap on packet count.

`Packets::try_from_abi_encoded` decodes every entry and `create_packets_attestation` then turns the resulting `Packets` into a `FuturesOrdered<get_commitment>` and awaits `collect::<Vec<_>>()`, which polls all chain queries concurrently against the configured adapter.

The only implicit ceiling is tonic's 4 MiB default decoded-message limit. A minimal `Packet` (short client IDs, a single payload, empty `value`) ABI-encodes to roughly 400-700 bytes, so one request can carry on the order of 5,000-10,000 packets. The attestor then turns each into one `get_commitment` call, all in flight at once.

As a result:

1. The chain RPC node is hit with thousands of concurrent membership/non-membership queries per request. EVM `eth_getProof`, Cosmos ABCI store queries, and Solana RPC calls are all rate-limited or quota-bound at the upstream provider, so saturating that budget either trips upstream rate limits (returning `RetrievalError` to honest callers) or stalls the attestor's adapter pool until requests time out.
2. The decoded `Packet` vector and the `FuturesOrdered` set both stay alive for the duration of the slowest chain query, multiplying the per-request memory footprint by `N`.

An unauthenticated client can repeat the request as quickly as the network allows, degrading availability for honest relayers without ever producing a valid attestation.

Recommendations:

Consider enforcing an explicit upper bound on `packets.len()` at the start of `packet_attestation`.

Cosmos: Resolved with [@45865ac9e5 ...](#)

Zenith: Verified.

4.3 Informational

A total of 2 informational findings were identified.

[I-1] Empty packet list bypasses commitment validation and reaches the signer

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [apps/ibc-attestor/src/rpc/attestor.rs#L164-L171](https://github.com/cosmos/ibc-attestor/blob/master/src/rpc/attestor.rs#L164-L171)

Description

There is no guard against an empty packet list in `create_packets_attestation`. An empty ABI payload (`packets = []`) decodes successfully, produces zero futures, and proceeds directly to `sign_attestation`:

```
let futures = packets          // len = 0
    .into_iter()
    .map(|packet| create_single_packet_attestation(...))
    .collect::<FuturesOrdered<_>>();

let validations = futures.collect::<Vec<_>>().await; // resolves
// immediately - []
let packets = validations.into_iter().collect::<Result<Vec<_>, _>>()?>; //
// Ok([])

// falls through to create_attestation and sign_attestation with packets =
// []
```

The result is a signed `PacketAttestation { height, packets: [] }`, which `AttestationLightClient.sol` immediately rejects on-chain with `EmptyPackets()`. No fraudulent attestation can be produced this way, but the signing operation has already been consumed.

Each empty `PacketAttestationRequest` costs: one `validate_height` RPC + one signing operation. Because the gRPC service has no authentication, any network-reachable party

can submit these requests freely.

AWS KMS enforces per-account cryptographic operation quotas. A sustained spray of empty requests exhausts these quotas, causing legitimate signing requests from valid relayers to fail. IFT transfers that depend on timely attestation are blocked until the quota resets.

The same pattern applies to `state_attestation`: any syntactically valid finalized height triggers one `validate_height` RPC, one `get_block_timestamp` RPC, and one signing operation. These attestations are similarly useless to a relayer but consume signer capacity.

Recommendations

We recommend rejecting empty packet lists at the entry point, before the height validation RPC is issued.

Cosmos: Acknowledged. We are not going to address this since a malicious client wanting to expend attester resources can still do so using a non empty packet list.

[I-2] Caller-controlled input errors are returned as internal service failures

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [apps/ibc-attestor/src/error.rs#L55-L74](#)
- [apps/ibc-attestor/src/adapter/mod.rs#L17-L35](#)
- [apps/ibc-attestor/src/adapter/cosmos.rs#L163-L172](#)
- [apps/ibc-attestor/src/lib.rs#L48-L55](#)

Description:

The `AttestorError` → `tonic::Status` conversion maps four variants uniformly to `Code::Internal`:

```
68: AttestorError::SignerError(_)
69: | AttestorError::SignerInitError(_)
70: | AttestorError::AbiError(_)
71: | AttestorError::AdapterError(_) => Self::new(Code::Internal,
    value.to_string()),
```

The first two are genuine service faults. The other two cover caller-induced cases:

`AbiError` originates in `Packets::try_from_abi_encoded` when the request payload fails ABI decoding, which is pure caller input:

```
48: fn try_from_abi_encoded(encoded: &[Vec<u8>]) -> Result<Self,
    AttestorError> {
49:     let packets = encoded
50:         .iter()
51:         .map(|p| Packet::abi_decode(p).map_err(AttestorError::AbiError))
52:         .collect::<Result<Vec<_>, _>>()?;
53:
54:     Ok(Self(packets))
55: }
```

`InvalidHeight`: caller-supplied height that is rejected.

Invalid client requests look like service faults, triggering unnecessary retries on the caller side and polluting alerting on the operator side.

Recommendations:

Consider refactoring the errors to distinguish caller input errors from actual, internal service faults.

Cosmos: Resolved with [@807e6238c1 ...](#)

Zenith: Verified.